CS405 Project 1 — Transformations, Projection & Curves from Paintings

Student: Göktuğ Kırkıl · *Student ID:* 33864

Painting: Hoca Ali Rıza — *Moda İskelesi* (Sakıp Sabancı Museum collection)

Repository: github.com/goktugkirkil/cs405-project1 · *Tag:* project1-final

1. The painting and the scene



Figure 1. Hoca Ali Rıza, Moda İskelesi.

I chose Moda İskelesi for two reasons. First, the location provides strong perspective cues which include the pier and the shore line and the horizon. These elements assist with filming through their utility. Second, the area displays distinct natural formations. The big leaning plane tree and the hill across the water create suitable shapes for tracing curves. The painting provides evidence which supports both Part C and Part D. The painting hangs in the scene as a textured quad in the plane z = 0. The object maintains its 900 × 737 aspect ratio because it measures 2 units high and 2.44 units wide. In front of it there are three objects (Part A) and a small harbour crane (Part B). Two curves are drawn on the painting (Part D). The camera orbits around the scene (Part C).

2. Part A — Transformations

Convention: In all my code, I use column vectors: p′ = M · p. A matrix is a Float32Array with 16 numbers. The numbers are stored column by column (column-major order), so the element in row r and column c is at index c·4 + r. WGSL mat4x4f uses the same layout, so I can upload my matrices directly, with no changes. multiply(A, B) returns A · B. This means B acts on the point first. In a product, the matrix on the right acts first.

Objects: There are three objects in front of the painting: a crate on the shore, a buoy on the water and an orange pyramid. The pyramid is controlled from the UI: translation, rotation axis and angle, and uniform scale.

Composition order: In the UI, you can switch between M = T·R·S and M = S·R·T. The screen shows both matrices. In my test, I used t = (0.55, −0.45, 0.45), a 30° rotation around z, and a scale of 0.25.

- In both orders, the upper-left 3×3 part is the same: 0.25 · Rz(30°). This is because R and S commute when the scale is uniform.
- The difference is in the translation column. With T·R·S, it is (0.55, −0.45, 0.45). These are exactly the slider values.
- With S·R·T, it is (0.18, −0.03, 0.11). Here the translation happens first, and after that it is also rotated and scaled: t′ = S·R·t.

So with T·R·S, the object is first scaled and rotated around its own centre, and then it is moved to its place. With S·R·T, the object is moved first. Because of this, the rotation turns it around the world origin, and the scale also makes its position smaller. T·R·S is the order we usually want. The rest of my code (all objects and the scene graph) uses T·R·S.





Figure 2. Left: M = T·R·S. Right: M = S·R·T with the same slider values. Only the translation column changes, and the pyramid moves.

Rigid inverse: If M = [R | t] (only rotation and translation), its inverse is $M^{-1} = [R^T \mid -R^T t]$. R^T is equal to R^{-1} because the columns of R are orthonormal. This is cheaper than a general inverse. I use it again for the view matrix in Part C.

3. Part B — Scene graph

Each node has a local matrix for its own, a parent, children and a world matrix. The world matrix is computed by a depth-first traversal(DFS) from the root:

$$\text{world}(\text{node}) = \text{world}(\text{parent}) \cdot \text{local}(\text{node}), \quad \text{world}(\text{root}) = \text{local}(\text{root})$$

The scene graph is a harbour crane. It has 7 nodes and 4 levels (crane → jib → hook → cargo). The jib turns around the tower (rotateY), and the cargo spins on its cable. When the jib turns, the hook, the cable and the cargo move with it. This shows that child nodes move together with their parent nodes.

Node (level)	Mesh	Local transform
crane (0, root)	—	T(0.98, −0.82, 0.45)
tower (1)	cube	T(0, 0.32, 0) · S(0.05, 0.64, 0.05)
jib (1)	—	T(0, 0.64, 0) · Ry(0.6 t) animated
jibBeam (2)	cube	T(0.17, 0, 0) · S(0.50, 0.035, 0.035)
hook (2)	—	T(0.38, 0, 0)
cable (3)	cube	T(0, −0.09, 0) · S(0.008, 0.18, 0.008)
cargo (3)	cube	T(0, −0.22, 0) · Ry(−1.5 t) · S(0.09) animated



Figure 3. Debug mode: the local axes of every node are drawn (red x, green y, blue z). The crane is on the right.

Hand check of a leaf: I paused the animation and pressed "reset scene", so t = 0 and both rotations are the identity. Then the world matrix of the leaf node "cargo" is W_cargo = M_crane · M_jib · M_hook · M_cargo

$$W = T(0.98, -0.82, 0.45) \cdot T(0, 0.64, 0) \cdot T(0.38, 0, 0) \cdot T(0, -0.22, 0) \cdot S(0.09)$$

The four translations add up: x = 0.98 + 0.38 = 1.36, y = −0.82 + 0.64 − 0.22 = −0.40, z = 0.45. The scale stays on the diagonal. So:

$$\begin{vmatrix} 0.09 & 0 & 0 & 1.36 \\ 0 & 0.09 & 0 & -0.40 \\ 0 & 0 & 0.09 & 0.45 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

The “cargo.world (leaf)” panel in the app shows the same matrix at t = 0

4. Part C — Camera and projection

4.1 View matrix (lookAt)

The view matrix takes the world and puts it into camera space. In camera space the camera sits at the origin, looks down −z, and +y is up. To get the camera axes, I use two cross products:

1. Forward: f = normalize(target − eye).
2. Right: s = normalize(f × up). This is the first cross product.
3. True up: u = s × f, the second one. Since s and f are already unit length and perpendicular to each other, u comes out as a unit vector too, so I don't need to normalize it.

If we think of the camera as an object in the world, its world matrix C has the columns s, u, −f and eye. I use −f instead of f because the camera looks down −z. The view matrix is just the inverse of C. C only has rotation and translation, so I can use the rigid inverse from Part A again:

$V = C^{-1} = [R^T \mid -R^T \cdot eye]$

So the rows of V are s, u and −f, and the translation part is (−s·eye, −u·eye, f·eye). In the code, lookAt just builds C and returns invertRigid(C).

Degenerate case: There is one problem case: when up is parallel to f. Then f × up = 0, and normalize gives a zero vector. In practice this happens when the camera looks straight up or straight down. To catch it, I check if |f · up| > 0.999. If it is, I switch to another up vector: (0, 0, 1), or (1, 0, 0) when f is close to the z axis.

4.2 Perspective projection for WebGPU

WebGPU expects depth to be in the range [0, 1] after the divide by w. OpenGL uses [−1, 1] instead, so I can't just copy the OpenGL matrix here. The last row of my matrix is (0, 0, −1, 0), which means w = −z. For depth, I write:

$$z_{ndc} = (A \cdot z + B) / (-z)$$

Then I set two conditions: a point at z = −near should get depth 0, and a point at z = −far should get depth 1.

$$(-A \cdot n + B) / n = 0 \Rightarrow B = A \cdot n$$
$$(-A \cdot f + A \cdot n) / f = 1 \Rightarrow A = f / (n - f),\ B = n \cdot f / (n - f)$$

For x and y, I use F = 1 / tan(fovY / 2). I multiply y by F and x by F / aspect. Dividing by the aspect ratio keeps the image from looking stretched on a wide screen.

$$\begin{vmatrix} F/aspect & 0 & 0 & 0 \\ 0 & F & 0 & 0 \\ 0 & 0 & f/(n-f) & n \cdot f/(n-f) \\ 0 & 0 & -1 & 0 \end{vmatrix}$$

4.3 Orthographic projection

In the orthographic case, w is always 1, so the divide by w does not change anything. The matrix only scales and moves a box. x goes from [l, r] to [−1, 1], with a scale of 2/(r − l) and an offset of −(r + l)/(r − l). y works the same way, with b and t. For depth, I use z_ndc = A·z + B with the same two conditions as before, and this gives A = 1/(n − f) and B = n/(n − f). In the app, the height of the ortho box is radius · tan(fov/2). This way, both projections show the scene at the same size.



Figure 4. The same view with perspective (left) and orthographic projection (right).

4.4 Orbit camera, inspection camera and frustum

The main camera moves on a sphere around its target. When you drag the mouse, yaw and pitch change, and the mouse wheel changes the radius. The eye position is:

$$\text{eye} = \text{target} + r\cdot(\cos p \cdot \sin y,\ \sin p,\ \cos p \cdot \cos y)$$

The panel shows the eye, target, fov, near and far values. There is also a second camera, the inspection camera. It doesn't move and looks at the scene from the side. From this camera, I draw the frustum of the main camera. I compute its 8 corners in camera space first. At distance d, the half height is d·tan(fov/2), and for ortho I use a fixed half height. Then I move the corners to the world with invertRigid(V).

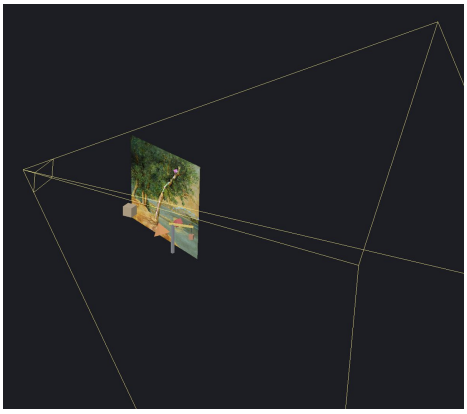


Figure 5. The main camera frustum (yellow) seen from the inspection camera. The small rectangle is the near plane.

4.5 Near, far and depth precision

I use near = 0.5 and far = 8. The whole scene is about 3.5 to 5 units away from the camera, so it fits inside this range. But perspective depth is not linear. With these values, a point at distance d gets the depth $1.067 - 0.533/d$. This means half of the depth range (0 to 0.5) is used only for distances between 0.5 and 0.94. The whole scene only gets depth values between about 0.91 and 0.96.

With near = 0.05 and far = 30, it gets much worse. Half of the range is already used before distance 0.1, and the whole scene is squeezed between 0.987 and 0.992. In this case, two surfaces that are close to each other can end up with the same depth value. This is called z-fighting. So near should be as large as the scene allows. Making near too small hurts more than making far large. In orthographic projection, depth is linear, so this problem doesn't show up in the same way. I didn't see any visible z-fighting in this small scene, because the objects are not very close to the painting surface.

5. Part D — Curves over the painting

I traced two shapes on the painting. The first one is a piecewise cubic Bézier curve with 3 segments. It follows the trunk and the main branch of the big leaning plane tree. The second one is a uniform Catmull–Rom spline, and it follows the outline of the hill on the other side of the water. All control points lie in the plane of the painting. To pick them, I turn the mouse position into a ray from the camera and intersect this ray with the plane. You can drag the points, and shift + click adds a new segment (for the Bézier) or a new point (for the Catmull–Rom).

5.1 Bernstein basis and end points

A cubic Bézier curve is a weighted average of four control points:

$$B(t) = B_0(t)\cdot P_0 + B_1(t)\cdot P_1 + B_2(t)\cdot P_2 + B_3(t)\cdot P_3,\ t \in [0, 1]$$

$$B_0 = (1-t)^3,\ B_1 = 3t(1-t)^2,\ B_2 = 3t^2(1-t),\ B_3 = t^3$$

The weights always add up to 1. This is because they are the four terms of $((1 - t) + t)^3$, which is equal to 1.

End points: At $t = 0$, $B_0 = 1$ and $B_1 = B_2 = B_3 = 0$, so $B(0) = P_0$. At $t = 1$, $B_3 = 1$ and all the others are 0, so $B(1) = P_3$. So the curve starts at P_0 and ends at P_3. In general, it does not go through P_1 and P_2. These two points only pull the curve towards them.

5.2 de Casteljau construction

The de Casteljau algorithm finds the same point, but it only uses linear interpolation. At each level, I take the point at ratio t between every two neighbouring points. So 4 points give 3 new points, then 2, and then 1. This last point is B(t). In the app, the in-between levels are drawn in blue and green, and B(t) is shown as a red cross. You can change t with a slider.

5.3 C^1 continuity at a joint

Let segment 1 be $P_0..P_3$ and segment 2 be $Q_0..Q_3$, with $Q_0 = P_3$. The derivative of a Bézier curve at its ends is:

$$B'(1) = 3(P_3 - P_2),\ Q'(0) = 3(Q_1 - Q_0)$$

C^1 means that these two vectors are equal. Since $Q_0 = P_3$, this gives:

$$Q_1 = 2\cdot P_3 - P_2$$

So the joint must be exactly in the middle of the two handles next to it. In the app, when I drag a handle, the handle on the other side of the joint moves to its mirror position. When I drag a joint, both of its handles move together with it. This way, the curve stays C^1 while I edit it.

5.4 Catmull–Rom and the object on the curve

Unlike a Bézier curve, the Catmull–Rom spline goes through its control points. Between p_1 and p_2, it uses p_0 and p_3 as neighbours, and the tangent at p_1 is $(p_2 - p_0)/2$. Because of this, it is easy to put the points directly on the hill line.

Moving along the curve:

A small pyramid moves along the Bézier curve. Its position is B(t). Its direction comes from the tangent:

$$B'(t) = 3[(1-t)^2(P_1-P_0) + 2t(1-t)(P_2-P_1) + t^2(P_3-P_2)]$$

I compute the angle with atan2 and build $M = T \cdot Rz(angle) \cdot S$. This way, the pyramid always points along the curve. The curve can also be exported as JSON with normalized image coordinates (see Appendix B).



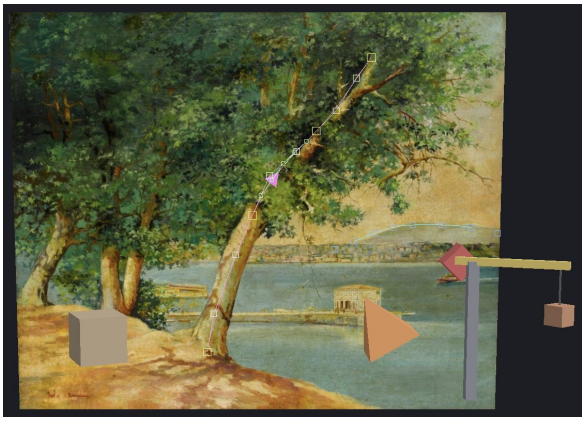


Figure 6. Left (zoomed): de Casteljau construction on segment 1 at t = 0.50. The levels are green and B(0.5) = (0.553, 0.376) is the red cross. Right: the traced Bézier on the tree, the Catmull–Rom spline on the hill, and the pink pyramid that follows the Bézier curve.

6. Integration and performance

Everything runs on a single page. In the panel, you can switch the view, change the projection, reset the scene, and turn the debug drawings (axes, frustum and curves) on or off. The code is split into four folders: src/math (maths and tests), src/scene (scene content and cameras), src/gpu (WebGPU and shaders) and src/ui (the panel). The README explains how to run the app and what each control does. There are unit tests for all the maths functions. You can run them with npm test, and they also run inside the app.

Measurement	Value
GPU	Apple M5 Pro (integrated GPU)
Browser	Google Chrome 156.0.8078.81 (arm64), macOS
Canvas	3024 × 1610 pixels
Frame time	8.33 ms (120 fps)
Vertices per frame	862
Draw calls per frame	10

The frame time is exactly 1000 / 120 ms. This is not a coincidence. My screen refreshes at 120 Hz, and requestAnimationFrame runs once for every refresh (vsync). So the 8.33 ms shows the limit of the display, not the cost of my scene. The real GPU work takes less time than this. The scene is very light: it has only 862 vertices and 10 draw calls. The CPU work in each frame (scene graph matrices and curve sampling in JavaScript) is also small.

7. What went wrong and how I found it

- The Bézier curve jumped far away from its control points, but all unit tests still passed. The problem was in the y component: I used the weight B_1 for P_0 instead of B_0. In my test, P_0 was (0, 0), so this term was always zero and the bug stayed hidden. I added a new test with $P_0 \neq 0$, and another check that bezier(t) gives the same point as de Casteljau. The test failed, I fixed the weight, and after that the curve was correct. The lesson: test data with zeros can hide bugs.
- In my first translate matrix, I put tx, ty and tz in the last row, because I wrote the matrix row by row. The column-major test failed. From this I learned that in the array, each group of four numbers is one column, not one row.
- A few times, the page stayed empty. The reason was an extra or a missing “}” after I edited a function. The Vite error overlay and npm test showed me the line number.
- At the start, npm crashed with Node 26 from Homebrew. I installed Node 22 LTS instead, and then it worked.

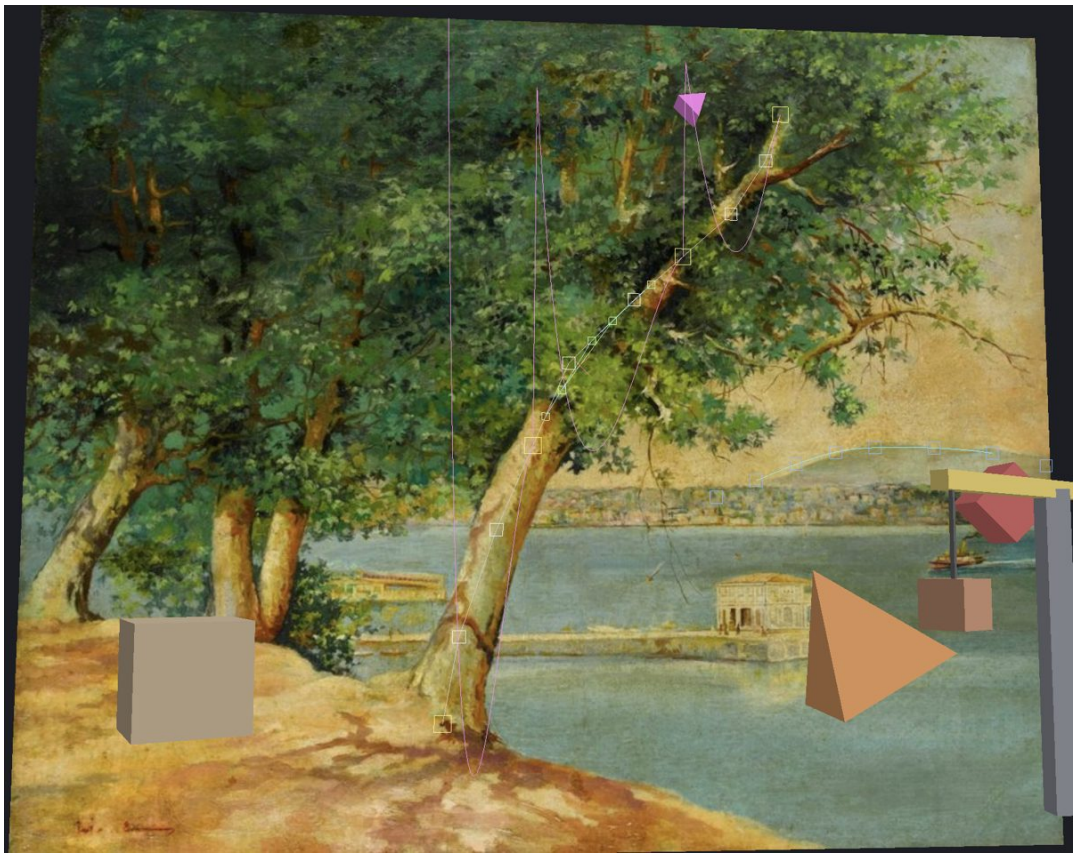


Figure 7. Before the fix: the wrong weight on P_0 made the Bézier curve jump far away from its control polygon.

Appendix A — Use of AI tools

Tool	What I asked	What I kept
Claude (Anthropic, claude.ai)	Set up the project, because the starter repository was not available to me: Vite, WebGPU context, painting texture (sRGB), render loop, frame-time counter. Build the parts that are not graded maths: UI panel, matrix display, mouse orbit, inspection view, frustum drawing, curve editing and picking, JSON export, scene content (object placement, crane), unit tests, README. Explain the maths for every graded function and check my attempts. Help me debug errors. Draft this report from my results and notes.	Kept as written by Claude: src/gpu/*, src/ui/panel.js, src/main.js, src/scene/{geometry, partA, partB, partD, picking, painting}.js, the helper code in src/scene/camera.js, src/math/tests.js, run-tests.js, README.md. Graded maths (vec3.js, mat4.js, compose.js, graph.updateWorld, camera.orbitEye / frustumCorners, curves.js): Claude gave the formulas and code skeletons with blanks. I filled the blanks and fixed the mistakes it pointed out. Given complete by Claude as a pattern example: vec3.add, mat4.identity, compose.composeTRS, and the x line of orbitEye, bezierTangent and catmullRom (I wrote the other components). Given as a complete corrected version after my own attempt: vec3 scale, dot, cross, length, normalize; mat4 translate, rotateX (layout), transformPoint, orthographic (layout); the −f / eye rows of lookAt; graph.updateWorld; curves.c1Partner and bezierTangent. Claude fixed syntax errors in my files (extra or missing braces, a duplicate function) and moved my compose functions to the correct file. Claude drafted a first version of the report from my results and notes. I rewrote every section in my own words and checked all derivations.

Appendix B — Exported curve (curve.json)

Normalised image coordinates: u = x / width (0 = left, 1 = right), v = y / height (0 = top, 1 = bottom).


```
{
  "painting": { "artist": "Hoca Ali Rıza", "title": "Moda İskelesi" },
  "bezier": {
    "type": "piecewise cubic Bézier, C1 at the joints", "segments": 3,
    "controlPoints": [ [0.395, 0.85], [0.41, 0.75], [0.455, 0.6], [0.49, 0.5],
      [0.525, 0.4], [0.58, 0.34], [0.62, 0.29],
      [0.66, 0.24], [0.7, 0.16], [0.73, 0.11] ] },
  "catmullRom": {
    "type": "uniform Catmull–Rom",
    "points": [ [0.66, 0.585], [0.7, 0.565], [0.74, 0.545], [0.78, 0.53],
      [0.82, 0.524], [0.88, 0.524], [0.94, 0.53], [0.995, 0.545] ] }
}
```